

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Метод Гаусса»

Выполнил:

Студент 3825Б1ПМии1

Катютин З. В.

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Введение.....	3
Постановка задачи.....	4
Руководство пользователя	5
Описание программной реализации	7
Результаты экспериментов	13
Заключение.....	15
Литература	16
Приложение	17

Введение

Метод Гаусса является фундаментальным алгоритмом линейной алгебры, используемым для решения систем линейных уравнений, определения ранга матрицы и вычисления определителя. Он основан на последовательном исключении переменных из уравнений (приведению к ступенчатому виду) с помощью элементарных преобразований строк, таких как перестановка строк, умножение строки на ненулевую константу и прибавление к строке другой строки, умноженной на константу. В данной лабораторной работе реализован метод Гаусса с частичным выбором ведущего элемента (поиск максимального по модулю элемента в текущем столбце).

Постановка задачи

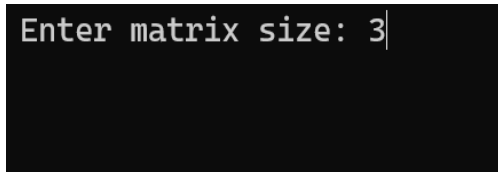
Разработать программу для решения системы линейных алгебраических уравнений с действительной квадратной матрицей методом Гаусса с выбором ведущего элемента.

Программа должна обеспечивать ввод размерности системы, коэффициентов матрицы и вектора свободных членов, после чего выполнять вычисление неизвестных и выводить результат на экран. Алгоритм должен включать прямой ход с частичным выбором ведущего элемента (поиск максимального по модулю элемента в столбце и перестановку строк) для повышения численной устойчивости, а также обратный ход для нахождения корней. В программной реализации необходимо использовать шаблоны для обеспечения работы с различными типами данных, а также реализовать структуры для хранения векторов и матриц с возможностью доступа по индексу и изменения размера.

Руководство пользователя

При запуске программы активируется пользовательский интерфейс. Программа предлагает пользователю ввести размерность системы линейных уравнений.

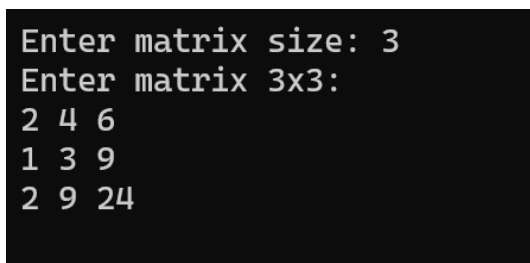
Пользователь должен ввести целое число n — количество уравнений в системе:



```
Enter matrix size: 3|
```

Рисунок 1 Ввод размера матрицы

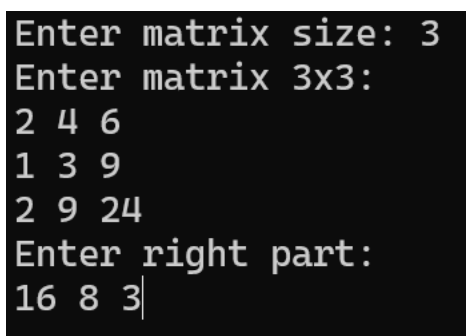
После ввода размерности программа запрашивает ввод элементов квадратной матрицы построчно. Пользователь должен последовательно ввести n строк, каждая из которых содержит n действительных чисел, разделенных пробелами:



```
Enter matrix size: 3
Enter matrix 3x3:
2 4 6
1 3 9
2 9 24
```

Рисунок 2 Ввод элементов квадратной матрицы

После заполнения матрицы программа запрашивает ввод правой части системы. Пользователь должен ввести n действительных чисел (значений свободных членов), разделенных пробелами или переводами строки.



```
Enter matrix size: 3
Enter matrix 3x3:
2 4 6
1 3 9
2 9 24
Enter right part:
16 8 3|
```

Рисунок 3 Ввод значений свободных членов

После выполнения всех предыдущих действий в консоль выводится результат решения системы — значения неизвестных $x_1, x_2, \dots, x_n$. Вывод имеет следующий формат:

```
Result:  
x1 = 17.75  
x2 = -6.5  
x3 = 1.08333
```

Рисунок 4 Вывод результатов

Описание программной реализации

Моя программа состоит из трех файлов заголовков (`Vector.h` , `SquareMatrix.h` , `SLAU.h`), содержащих объявления шаблонных классов и их методов, и одного исходного файла (`методГаусса.cpp`), содержащего функцию `main()` .

1. Файловая структура программы

- **`Vector.h`** – содержит шаблонный класс `Vector` для работы с одномерными динамическими массивами.
- **`SquareMatrix.h`** – содержит класс квадратной матрицы `SquareMatrix` .
- **`SLAU.h`** – содержит класс `SLAU` (наследник `SquareMatrix`) с реализацией метода Гаусса.
- **`методГаусса.cpp`** – содержит функцию `main()` и организует пользовательский ввод вывод.

2. Классы программы

2.1. Класс `Vector<T>` (файл `Vector.h`)

Предоставляет интерфейс для работы с одномерным динамическим массивом элементов произвольного типа.

Принцип работы: Класс является оберткой над `std::vector<T>` , скрывая детали реализации.

Методы класса:

1) `Vector(int n = 0)` (конструктор)

Входящие параметры: `n` (`int`, по умолчанию 0) – начальный размер вектора.

Возвращаемое значение: нет (конструктор).

Принцип работы: Создает внутренний `std::vector<T>` размером `n` , все элементы инициализируются значением по умолчанию для типа `T` .

2) `int size() const`

Входящие параметры: нет.

Возвращаемое значение: `int` – текущее количество элементов в векторе.

Принцип работы: Вызывает метод `size()` у внутреннего вектора `data` и возвращает результат.

3) `T& operator[](int i)`

Входящие параметры: `i (int)` – индекс элемента.

Возвращаемое значение: `T&` – ссылка на элемент с индексом `i` (позволяет изменять значение).

Принцип работы: Возвращает ссылку на `i`-й элемент внутреннего вектора `data`.

4) `const T& operator[](int i) const`

Входящие параметры: `i (int)` – индекс элемента.

Возвращаемое значение: `const T&` – константная ссылка на элемент с индексом `i` (только для чтения).

Принцип работы: Возвращает константную ссылку на `i`-й элемент внутреннего вектора `data`.

5) `void resize(int n)`

Входящие параметры: `n (int)` – новый размер вектора.

Возвращаемое значение: нет.

Принцип работы: Вызывает метод `resize(n)` у внутреннего вектора `data`.

2.2. Класс `SquareMatrix<T>` (файл `SquareMatrix.h`)

Представляет квадратную матрицу и предоставляет базовые операции.

Принцип работы: Матрица реализована как «вектор из векторов» – внешний вектор хранит строки, каждая строка является вектором элементов.

Поля класса:

- `Vector<Vector<T>> matrix` – хранилище данных матрицы
- `int n` – размерность матрицы

Методы класса:

1) `SquareMatrix(int size = 0)` (конструктор)

Входящие параметры: `size (int, по умолчанию 0)` – размерность квадратной матрицы.

Возвращаемое значение: `нет`.

Принцип работы: Сохраняет `size` в поле `n`, изменяет размер внешнего вектора `matrix` до `n`, затем в цикле изменяет размер каждого внутреннего вектора (строки) до `n`.

2) `int size() const`

Входящие параметры: `нет`.

Возвращаемое значение: `int` – размерность матрицы.

Принцип работы: Возвращает значение поля `n`.

3) `Vector<T>& operator[] (int i)`

Входящие параметры: `i (int)` – индекс строки.

Возвращаемое значение: `Vector<T>&` – ссылка на строку матрицы с индексом `i`.

Принцип работы: Возвращает `i`-й элемент вектора `matrix` (который является вектором строкой).

4) `const Vector<T>& operator[] (int i) const`

Входящие параметры: `i (int)` – индекс строки.

Возвращаемое значение: `const Vector<T>&` – константная ссылка на строку матрицы с индексом `i`.

Принцип работы: Константная версия оператора `[]`.

5) `void input()`

Входящие параметры: `нет` (данные читаются из консоли).

Возвращаемое значение: `нет`.

Принцип работы: Выводит приглашение "Enter matrix n x n:", затем в двойном цикле считывает из стандартного ввода значения в `matrix[i][j]`.

6) `void output() const`

Входящие параметры: `нет`.

Возвращаемое значение: `нет`.

Принцип работы: В двойном цикле выводит все элементы матрицы в консоль, после каждой строки переводит курсор на новую строку.

7) `void swapRows(int row1, int row2)`

Входящие параметры: `row1 (int)` – индекс первой строки, `row2 (int)` – индекс второй строки.

Возвращаемое значение: нет.

Принцип работы: Если `row1 == row2`, функция завершается без действий. В противном случае в цикле по столбцам от 0 до `n-1` меняет местами `matrix[row1][j]` и `matrix[row2][j]` через временную переменную.

2.3. Класс **SLAU<T>** (файл **SLAU.h**)

Представляет систему линейных алгебраических уравнений. Наследует `SquareMatrix<T>`.

Принцип работы: Класс наследует все свойства квадратной матрицы (коэффициенты системы) и добавляет метод для решения СЛАУ методом Гаусса с выбором ведущего элемента.

Методы класса:

1) **SLAU(int size)** (конструктор)

Входящие параметры: `size (int)` – размерность системы.

Возвращаемое значение: нет.

Принцип работы: Вызывает конструктор базового класса `SquareMatrix<T>(size)`, создавая квадратную матрицу коэффициентов размера `size`.

2) **Vector<T> gauss(Vector<T>& rightPart)**

Входящие параметры: `rightPart (Vector<T>&)` – ссылка на вектор правой части системы (размером `n`).

Возвращаемое значение: `Vector<T>` – вектор решение системы (размером `n`).

Принцип работы: Решает систему методом Гаусса с частичным выбором ведущего элемента. Алгоритм описан ниже.

3. Описание алгоритма метода Гаусса (метод **SLAU<T>::gauss**)

Решение системы линейных уравнений $Ax = b$, где A – квадратная матрица коэффициентов, b – вектор правой части.

Входные данные: Вектор `rightPart` (правая часть системы). Матрица коэффициентов хранится в унаследованном поле `matrix`.

Выходные данные: Вектор `result` (решение системы).

Алгоритм:

Шаг 1. Прямой ход (приведение треугольному виду)

Для каждого столбца `k` от 0 до `n-2` выполняется:

1. Поиск ведущего элемента:

В столбце `k` среди строк от `k` до `n-1` ищется строка `maxRow`, содержащая максимальный по модулю элемент.

`maxVal = max(|matrix[i][k]|)` , где `i = k..n-1`

2. Перестановка строк:

Если `maxRow != k`, то:

- Вызывается `swapRows(k, maxRow)` – строки матрицы меняются местами
- Элементы `rightPart[k]` и `rightPart[maxRow]` обмениваются значениями

3. Исключение переменных:

Для каждой строки `i` от `k+1` до `n-1`:

- Вычисляется коэффициент `factor = matrix[i][k] / matrix[k][k]`
- Для каждого столбца `j` от `k` до `n-1`: `matrix[i][j] -= factor * matrix[k][j]`
- `rightPart[i] -= factor * rightPart[k]`

Шаг 2. Обратный ход (нахождение неизвестных)

Создается пустой вектор `result` размером `n`.

Для `i` от `n-1` до 0 (с конца к началу):

1. Вычисляется сумма: `sum = 0`

Для `j` от `i+1` до `n-1`: `sum += matrix[i][j] * result[j]`

2. Вычисляется значение неизвестной:

`result[i] = (rightPart[i] - sum) / matrix[i][i]`

Шаг 3. Возврат результата

Метод возвращает вектор `result`.

4. Функция `main()` (файл `методГаусса.cpp`)

Точка входа в программу. Организует ввод данных, вызов метода Гаусса и вывод результата.

Входящие параметры: нет (данные читаются из консоли).

Возвращаемое значение: `int` – 0 при успешном завершении.

Принцип работы и алгоритм:

1. Объявляет переменную `n` (`int`).
2. Выводит сообщение "Enter matrix size:" и считывает значение `n` из консоли.
3. Создает объект `slae` класса `SLAU<double>(n)`.
4. Вызывает метод `slae.input()` – пользователь вводит матрицу коэффициентов.
5. Создает вектор `rightPart` размером `n`.
6. Выводит сообщение "Enter right part:" и в цикле считывает `n` чисел в `rightPart`.
7. Вызывает `slae.gauss(rightPart)` и сохраняет результат в векторе `result`.
8. Выводит сообщение "Result:" и в цикле выводит каждую неизвестную в формате "`xi = значение`".
9. Возвращает 0.

Результаты экспериментов

После того как я добился успешной работы моей программы, мною было проведено несколько экспериментов. Целью экспериментов являлась проверка корректности реализации метода Гаусса с выбором ведущего элемента, а также оценка его численной устойчивости на системах различной размерности и с разными типами матриц.

Эксперимент 1. Система размерности 3×3

Рассмотрим систему линейных уравнений:

$$3x_1 + 2x_2 - x_3 = 1$$

$$2x_1 - 2x_2 + 4x_3 = -2$$

$$-x_1 + 0.5x_2 - x_3 = 0$$

Точное решение системы (найдено аналитически):

$$x_1 = 1, \quad x_2 = -2, \quad x_3 = -2$$

Входные данные программы:

```
Enter matrix size: 3
```

```
Enter matrix 3x3:
```

```
3 2 -1
```

```
2 -2 4
```

```
-1 0.5 -1
```

```
Enter right part:
```

```
1 -2 0
```

Результат работы программы:

```
Result:
```

```
x1 = 1
```

```
x2 = -2
```

```
x3 = -2
```

Вывод: Решение, полученное программой, полностью совпадает с аналитическим.

Эксперимент 2. Система размерности 2×2

Рассмотрим систему:

$$2x_1 + 3x_2 = 8$$

$$5x_1 - x_2 = 3$$

Точное решение:

$$x_1 = 1, \quad x_2 = 2$$

Результат работы программы:

Result:

$$x1 = 1$$

$$x2 = 2$$

Вывод: Программа корректно решает систему 2×2.

Заключение

Я написал рабочую программу на языке C++, которая успешно решает системы линейных алгебраических уравнений методом Гаусса с выбором ведущего элемента. Были реализованы шаблонные классы `Vector` для работы с одномерными массивами, `SquareMatrix` для представления квадратной матрицы, а также класс `SLAU`, наследующий `SquareMatrix`, содержащий реализацию метода Гаусса. Был реализован выбор ведущего элемента для повышения численной устойчивости алгоритма, а также функция `swapRows` для перестановки строк матрицы и соответствующих элементов правой части. Программа предоставляет удобный интерфейс для ввода размерности системы, матрицы коэффициентов и вектора свободных членов, после чего выводит полученное решение. По результатам экспериментов я сделал вывод о корректности реализации метода Гаусса и эффективности выбора ведущего элемента при решении систем с матрицами, имеющими малые диагональные элементы.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.
3. Бахвалов Н. С., Жидков Н. П., Кобельков Г. М. Численные методы. – 8-е изд. – М.: Лаборатория знаний, 2015.
4. Вержбицкий В. М. Численные методы. Линейная алгебра и нелинейные уравнения: Учебное пособие для вузов. — М.: Высшая школа, 2000.

Приложение

```
template<typename T>
class SLAU : public SquareMatrix<T>{
public:
    SLAU(int size) : SquareMatrix<T>(size) {}

    //метод гаусса
    Vector<T> gauss(Vector<T>& rightPart){
        int n = this->n;

        //прямой ход
        for (int k = 0; k < n - 1; k++){
            //Выбор ведущего элемента
            int maxRow = k;
            T maxVal = std::abs(this->matrix[k][k]);
            for (int i = k + 1; i < n; i++) {
                if (std::abs(this->matrix[i][k]) > maxVal){
                    maxVal = std::abs(this->matrix[i][k]);
                    maxRow = i;
                }
            }
            //swap строк
            if (maxRow != k){
                this->swapRows(k, maxRow);
                T temp = rightPart[k];
                rightPart[k] = rightPart[maxRow];
                rightPart[maxRow] = temp;
            }

            //исключение
            for (int i = k + 1; i < n; i++) {
                T factor = this->matrix[i][k] / this->matrix[k][k];
                for (int j = k; j < n; j++) {
                    this->matrix[i][j] -= factor * this->matrix[k][j];
                }
                rightPart[i] -= factor * rightPart[k];
            }
        }

        //обратный ход
        Vector<T> result(n);
        for (int i = n - 1; i >= 0; i--){
            T sum = 0;
            for (int j = i + 1; j < n; j++) {
                sum += this->matrix[i][j] * result[j];
            }
            result[i] = (rightPart[i] - sum) / this->matrix[i][i];
        }

        return result;
    }
};
```

Рисунок 5 Реализация метода Гаусса

```
int main(){
    int n;
    std::cout << "Enter matrix size: ";
    std::cin >> n;
    SLAU<double> slau(n);
    slau.input();

    Vector<double> rightPart(n);
    std::cout << "Enter right part:\n";
    for (int i = 0; i < n; i++){
        std::cin >> rightPart[i];
    }

    Vector<double> result = slau.gauss(rightPart);
    std::cout << "Result:\n";
    for (int i = 0; i < n; i++){
        std::cout << "x" << i + 1 << " = " << result[i] << "\n";
    }
    return 0;
}
```

Рисунок 6 Основная функция main()